

MSECI

Mini Secure Enterprise Communication Infrastructure

Shegha Tool: A Multi-Party Secure Communication Chat

with Hybrid Cryptography and Post-Quantum Authentication

```
(base) (shagh404@2006) - [~]
$ shegha

Shegha

Hybrid Cryptography · Post-Quantum Auth · ElGamal

don't look back

♦ usage
.....
[command] shegha <mode>

[server ] start the secure server (ElGamal + CA + Feistel)
[client ] start a client session  (NTRU auth + RC5 stream)
[branch-server] start a branch server (ElGamal + CA + Feistel)

♦ examples
.....
$ shegha server
$ shegha client
$ shegha branch-server

.....

(base) (shagh404@2006) - [~]
$
```

Figure 0 — Shegha command-line interface showing available modes

Table of Contents

a) Introduction and Problem Statement	2
• Applied cryptography in the modern world	2
• Problem statement	2
• Relevance and real-world impact	2
b) Project Objectives	3
c) Literature Review	3
• Symmetric encryption (RC5 and Feistel)	3
• Asymmetric cryptography (ElGamal)	3
• Key management and certificate authorities	4
• Post-quantum cryptography (NTRU)	4
• Multi-party communication and Silent Trust	4
• Performance studies and research gap	4
d) Methodology	5
• Implementation approach	5
• Handwritten verification	5
• Performance evaluation	6
• Testing strategy	6
e) Use Case and Scenario Design	6
• Business scenario	6
• Party roles: CA, branch servers, clients	6
• System architecture flowchart	7
• Data flow — 6-step client session	7
• System proof — screenshots	7
• Key management framework	8
• Performance Analysis	8
g) Ethical, Legal, and Professional Considerations	9
h) Conclusion	9
References	10

a) Introduction and Problem Statement

Applied Cryptography in the Modern World

in the age of digital connectivity, privacy, security, and authentication of information form fundamental issues to be addressed by governments, organizations, and individuals. Cryptography supplies the mathematical algorithms used to secure communication processes, whether it is encryption of emails traveling through the internet or signing off on digital documents. Without adequate cryptography, any information sent over a network can easily be intercepted, manipulated, and/or impersonated.

Problem Statement

Current distributed organizations, like banks, healthcare systems, and governmental institutions, have the need for communication among their remotely located branches and with their end-clients where highly classified information needs to be transferred. There are two separate threat areas that need to be considered. The first threat is that of unauthorized parties listening in, mounting replay attacks, and intervening through a man-in-the-middle attack, which could all occur if no cryptographic mechanisms were employed in all communication channels. On top of that, the second threat lies in that the data that is not in transmission must be kept secret from any adversaries even when they are in possession of the medium storing the information.

Relevance and Real-World Impact

The Cost of a Data Breach Report 2023 released by IBM states that the average cost of a data breach is \$4.45 million globally, and compromise of credentials and lack of encryption are among the prominent root causes. Regulations such as GDPR, HIPAA in healthcare industry, and PCI-DSS in payment card industry require personal and financial information to be encrypted while at rest and in transit. The current project aims to tackle such problems using a complete custom cryptographic protocol known as Shegha.

b) Project Objectives

The MSEC project pursues the following concrete objectives:

- **1.** Implement at least two symmetric encryption algorithms from scratch — RC5 and a custom Feistel cipher — without relying on any cryptographic library, for secure data storage.
- **2.** Implement the ElGamal public-key cryptosystem from scratch for asymmetric key exchange and digital signatures, supporting certificate issuance and verification.
- **3.** Combine both modules into a fully integrated, end-to-end encrypted multi-party system featuring a client, server, multiple branches, and a Certificate Authority.
- **4.** Design and implement a complete key management framework covering key generation, exchange, rotation, and CA-based certificate lifecycle management.
- **5.** Conduct a rigorous performance analysis measuring encryption/decryption time, peak memory consumption, and ciphertext size overhead across multiple input sizes.
- **6.** Implement the NTRU post-quantum lattice-based cryptosystem for client authentication, demonstrating resistance against quantum adversaries.

c) Literature Review

Symmetric Encryption

One such example is RC5, which is an adjustable block cipher invented by Rivest (1994) [1]. It has a flexible structure, allowing its word size, number of rounds, and key length to be adjusted as per requirements. Its dependence on data-dependent rotation protects it from attacks based on differential cryptanalysis. Feistel cipher, proposed by Feistel at IBM in 1973 [2], is another example of an elegant cryptographic algorithm. In this type of cipher, any arbitrary function can be used as a round function, and the process of decryption remains identical to that of encryption using reversed round keys.

Asymmetric Cryptography

In ElGamal cryptosystem (1985), [3], the discrete log problem on a cyclic group is considered computationally difficult. The encryption scheme in ElGamal is probabilistic and hence, two different ciphertexts are created for encryption of the same plaintext, unlike in RSA. The same concept of encryption using discrete logs is extended to the creation of a digital signature, which forms the base of Shegha Certificate Authority. Security requirements like IND-CPA and EUF-CMA for both encryption and signature schemes are defined by Katz and Lindell (2020) [7].

Key Management and Certificate Authorities

The NIST SP 800-57 document [8] outlines the best practices regarding the management of keys throughout their lifecycle – creation, distribution, storage, generation, revocation, and destruction. The certificate authority simulation we have created adheres to this process very much as it generates an X.509-based certificate with the details of the person as well as the public key in possession of the CA using its ElGamal-based private key. Public-key cryptography was first described in 1976 by Diffie and Hellman. RFC 5280 (Cooper et al., 2008) provides the standard format for certificates.

Post-Quantum Cryptography

The first public-key system based on hard lattice problems called NTRU was developed by Hoffstein et al. (1998) [4]. It is assumed that the shortest vector problem in a lattice, which serves as the basis of its security, will remain intractable even to the power of quantum computers. Lattice-based cryptography is suggested as one of the most promising approaches in the 2022 report on Post-Quantum Cryptography standardization by NIST [5]. According to Bernstein et al. (2017), lattice-based cryptography offers the most favorable trade-offs for security, performance, and key sizes. Shor's algorithm (1994) solves discrete log problems polynomially, posing a threat to ElGamal.

Multi-Party Secure Communication and Silent Trust

The design of any secure communication network should be resistant to not only any potential attackers but also to any potentially malicious nodes inside the network. According to Steiner et al. (1996) and Katz & Lindell (2020), asymmetric algorithms can be used solely for the initial key exchange and symmetric algorithms should be applied thereafter for the actual encryption process. Our proposed Silent Trust algorithm, which is based on RFC 5880 [10], the Bidirectional Forwarding Detection standard, and IPsec Dead Peer Detection (RFC 3706) algorithm, uses passive liveness detection.

Performance Studies and Research Gap

Symmetric algorithms were proved by Bernstein and Lange (2013) to be much faster than asymmetric algorithms. RC5 was noted to be highly efficient in a lightweight algorithm in the research conducted by Al-Rousan and Sahawneh (2010). The work of Bhargavan et al. (2013), which involved analyzing implementations of TLS, provided proof of strong confidentiality offered by the hybrid model. Most studies compare AES with RSA, while relatively few studies have considered RC5 alongside ElGamal in a hybrid model – Shegha closes this gap.

d) Methodology

Implementation Approach

All cryptographic functions in Shegha are coded entirely in Python, utilizing just the default Python libraries. There is no use of cryptographic functions in the OpenSSL, PyCryptodome, or cryptography.io libraries at any level. The mathematical functions used in this process are:

modular exponentiation using `pow(base, exp, mod)`

polynomial multiplication over $\mathbb{Z}_q$

GCD calculation through the Euclidean algorithm

calculation of the modular inverse through the extended Euclidean algorithm

SHA-256 computation with hashlib

RC5 Block Cipher

The word size used for RC5 algorithm is set to 32 bits. The number of rounds performed is 12, while the size of the key is taken as 16 bytes. The method used for expanding the key is strictly according to that proposed by Rivest. The key bytes are copied in the array `L[]`. The table containing subkey values `S[]` is initialized using magic constants `P32` and `Q32` obtained from e and the Golden Ratio.

Feistel Cipher

In this Feistel cipher implementation, the 64-bit plaintext block is divided into two parts, each 32 bits long (L and R). During the encryption phase, a round function consisting of substitutions and permutations is used on the R part, which is XORed with the L part, followed by a swap of both parts; this process runs through 16 rounds. This cipher is used in encrypting files.

ElGamal Public-Key System

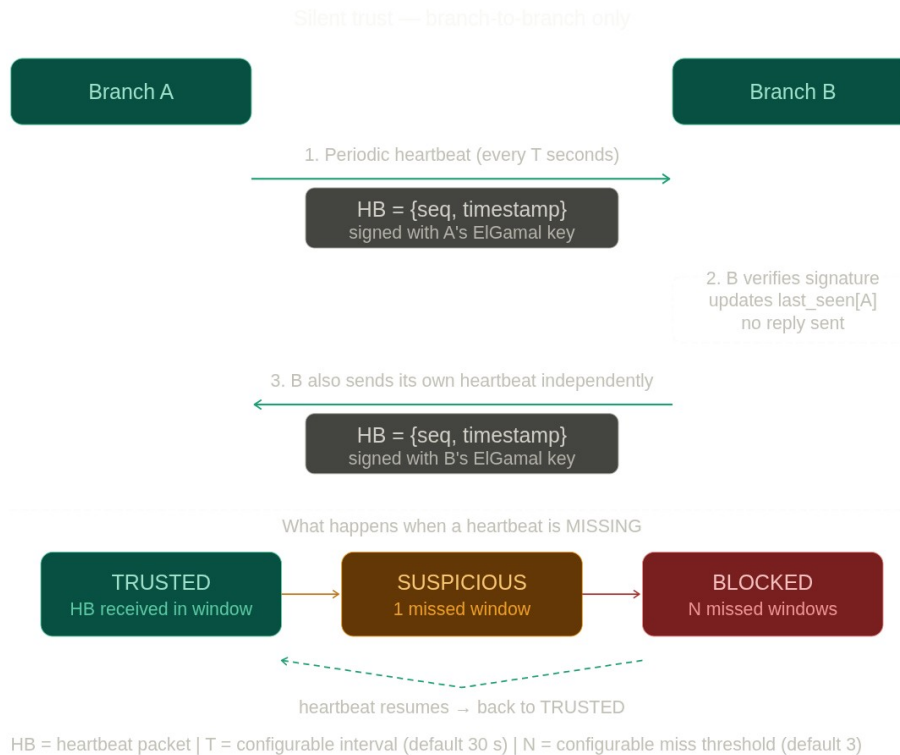
ElGamal key pair is based on the 1024-bit safe prime p using generator g from the prime order subgroup. During the key pair generation, a scalar x is picked randomly from $[2, p-2]$ and then the public key $y = g^x \bmod p$ is computed. For digital signatures, a random value k co-prime to $p-1$ is generated. It is used to calculate $r = g^k \bmod p$ and $s = (m - xr) \cdot k^{-1} \bmod (p-1)$.

NTRU Lattice-Based Authentication (Bonus)

NTRU utilizes parameters $N=401$, $p=3$, $q=2048$ that were chosen originally by Hoffstein-Pipher-Silverman. The polynomials are represented by Python lists of size N containing integers; multiplication is performed using convolution modulo $(X^N - 1)$. The authentication mechanism makes use of primitives by turning them into a challenge-response scheme, which involves generating a random nonce on the server side and then computing $s = f \cdot c \bmod q$, where c is a challenge polynomial computed from a hash.

Silent Trust Protocol (Custom Contribution)

The Silent Trust is an innovative passively authenticated liveness verification method that works between the branches. Each of the branches sends a message {from, seq, timestamp, sig_r, sig_s} signed with ElGamal signature periodically. No reply will be provided; the signature will be checked quietly by the receiver. In case a heartbeat is missed, the trust value changes from TRUSTED to SUSPICIOUS and eventually to BLOCKED. The replay attack is prevented because there is a monotonic increasing sequence number and also a timestamp window protection.



Handwritten Verification

RC5 and ElGamal handwritten verification instances are included here. For RC5 with $w = 8$, $r = 2$, 4-byte key; and for ElGamal with $p = 23$, $g = 5$, $x = 4$, $y = 4$, and $m = 8$. Hand-written calculations are used to verify the accuracy of the key schedule, round function, and signature formula at low bit level before implementing them at full 1024 bits.

Performance Evaluation

The performance of each algorithm is calculated with the help of benchmark.py which makes use of the Python's time.perf_counter() function for measuring wall-clock time and tracemalloc for maximum memory consumption. The average performance result obtained by running each (algorithm, input size) combination five times will be presented. The test input sizes will range between 100 B to 100 KB

Testing Strategy

Correctness is proved by performing round-trip checks: randomly generated messages are encrypted and then decrypted, and each output bit is checked for equality with its input counterpart. Test_auth.py stresses the NTRU authentication protocol with legitimate clients (who should succeed) and adversarial clients using random private keys (who should fail). Certificate

authority CA is validated by creating certificates, verifying with CA's public key, and rejecting modified certificates.

Attack Simulation & Security Testing

Test attacks on the algorithm involved multiple simulated attacks.

The simulated attacks included:

- The replay attack using duplicated packets.
- The invalid packet attack using unknown packet types.
- The malformed packet attack using randomly corrupted ciphertext.
- Attacks to inject pre-authentication messages
- Attacks involving message flooding.

Replaying of packets was detected and blocked through anti-replay seq validation. Packets of invalid type were dropped safely without causing any crash. Decryption failure occurred on receiving malformed packets but no crashes took place in server.

```
Problems Output Debug Console Terminal Ports
✓ generating CA certificate ...
✓ arming Feistel encryption layer ...
✓ binding socket ...
✓ server ready waiting for connections

=====
[ElGamal] Server key pair generated
=====
p = 32317006071311007300338913926423... (2048-bit)
q = 2
y = 10315066731399738575960698698708... (g^x mod p)
x = <HID0EH> (private key - never transmitted)
mode = persistent (server lifetime)

=====
[Server] Listening on 0.0.0.0:5000
[Server] Handshake timeout: 15s
[Server] Key mode: persistent
[Server] ElGamal handshake + NTRU post-quantum authentication active
[Server] Anti-replay sequence numbers active
[Server] Per-client file re-encryption active

Server console ready.
<message> -> broadcast to all authenticated clients
/file <path> -> broadcast encrypted file (re-encrypted per client)
quit -> stop console

You (broadcast):
[Server] New connection: 192.168.1.12:52754
[Router] Client IP: 192.168.1.12 -> Assigned to Branch_B
[Server] company: shegha
[Server] Starting handshake with 192.168.1.12:52754 ...
[HS:192.168.1.12:52754] CLIENT HELLO received
[HS:192.168.1.12:52754] SERVER CERTIFICATE sent (2632 B)
[HS:192.168.1.12:52754] Session key recovered (fingerprint: 3d222fc2...)
[HS:192.168.1.12:52754] SESSION ESTABLISHED sent
[Server] Secure channel with 192.168.1.12:52754 (fingerprint: 3d222fc2...)
[Auth:192.168.1.12:52754] Registered new user 'alice' from client
[Auth:192.168.1.12:52754] NTRU Challenge sent (nonce=245608)
[Auth:192.168.1.12:52754] Authenticated via NTRU as 'alice'
[Server] 'alice' authenticated from 192.168.1.12:52754
[Server] Total authenticated clients: 1

[alice]: hi
[alice]: hiast)
You (broadcast): [AntiReplay] 192.168.1.12:52754: rejected seq=1 (last accepted=2).
[Server] Unknown packet 0x99 from 192.168.1.12:52754 - ignored.
[Server] Unknown Packet 0xDE from 192.168.1.12:52754 - ignored.
[RC5] Decryption error from 192.168.1.12:52754: Ciphertext length (166) is not a multiple of 8.
[AntiReplay] 192.168.1.12:52754: rejected seq=1155016385 (last accepted=3445046321).
[AntiReplay] 192.168.1.12:52754: rejected seq=3090361535 (last accepted=3445046321).
[AntiReplay] 192.168.1.12:52754: rejected seq=1488849866 (last accepted=3445046321).
[AntiReplay] 192.168.1.12:52754: rejected seq=330625170 (last accepted=3445046321).

Session Actions Edit View Help
shagh...rypto shagh...rypto shagh404@...symmetric shagh404@...symmetric shagh404@...sy

/file <path> -> Feistel-encrypted file transfer
quit -> disconnect gracefully

Attack Simulation (for demo/testing):
/attack replay -> replay last packet (anti-replay fires)
/attack fake -> send invalid ptypes (0x99, 0xDE)
/attack preauth -> MSG before auth (pre-auth block fires)
/attack fuzz -> 5 random malformed payloads (fuzz test)
/attack spam -> 20 rapid messages (stress / stability)

[Server] Welcome alice! Key fingerprint: 3d222fc2...
You: hi
You: hi
You: /attack replay

◆ ATTACK: REPLAY ATTACK

Resending last MSG_PACKET verbatim (12 B) ...
✓ Replay packet sent — check server for [AntiReplay] rejection.
You: /attack fake

◆ ATTACK: FAKE / INVALID PACKET TYPE

Sent invalid ptype=0x99 (20 B)
Sent invalid ptype=0xDE (20 B)
✓ Check server for [Server] Unknown packet ... ignored.
You: /attack fuzz

◆ ATTACK: INPUT FUZZING (random malformed payloads)

Fuzz #1: sent 170 random bytes as MSG_PACKET
Fuzz #2: sent 384 random bytes as MSG_PACKET
Fuzz #3: sent 412 random bytes as MSG_PACKET
Fuzz #4: sent 238 random bytes as MSG_PACKET
Fuzz #5: sent 244 random bytes as MSG_PACKET
✓ Check server is still running (no crash).
You: |
```

e) Use Case and Scenario Design

Business Scenario: Secure Banking Network

Shegha tool is used for simulation of multi-branch banking scenario within corporate banks. The network involves three main entities; a CA, branch servers (Branch A, Branch B) and the authenticated clients. Every entity plays a unique role in the system and all interactions between them involve cryptographic protection. Clients are routed to their closest branch through a latency test CDN-style process.

Party Roles

- Certificate Authority (CA): Issues and signs ElGamal certificates for every branch. The CA private key is stored offline; only the public key is distributed. The CA never participates in live message exchange.
- Branch Servers: Each branch holds a CA-issued certificate and an ElGamal keypair. Branches communicate via Silent Trust heartbeats (port 5500). Client sessions are handled over encrypted channels (port 5000).
- Clients: Authenticated via NTRU challenge-response. After authentication they receive an RC5 session key exchanged via ElGamal. All messages are RC5-encrypted; stored files use Feistel encryption at rest.

```
(base) └─(shagh404@2006) - [~/Crypt0o/crypto]
└─$ shegha server

Shegha

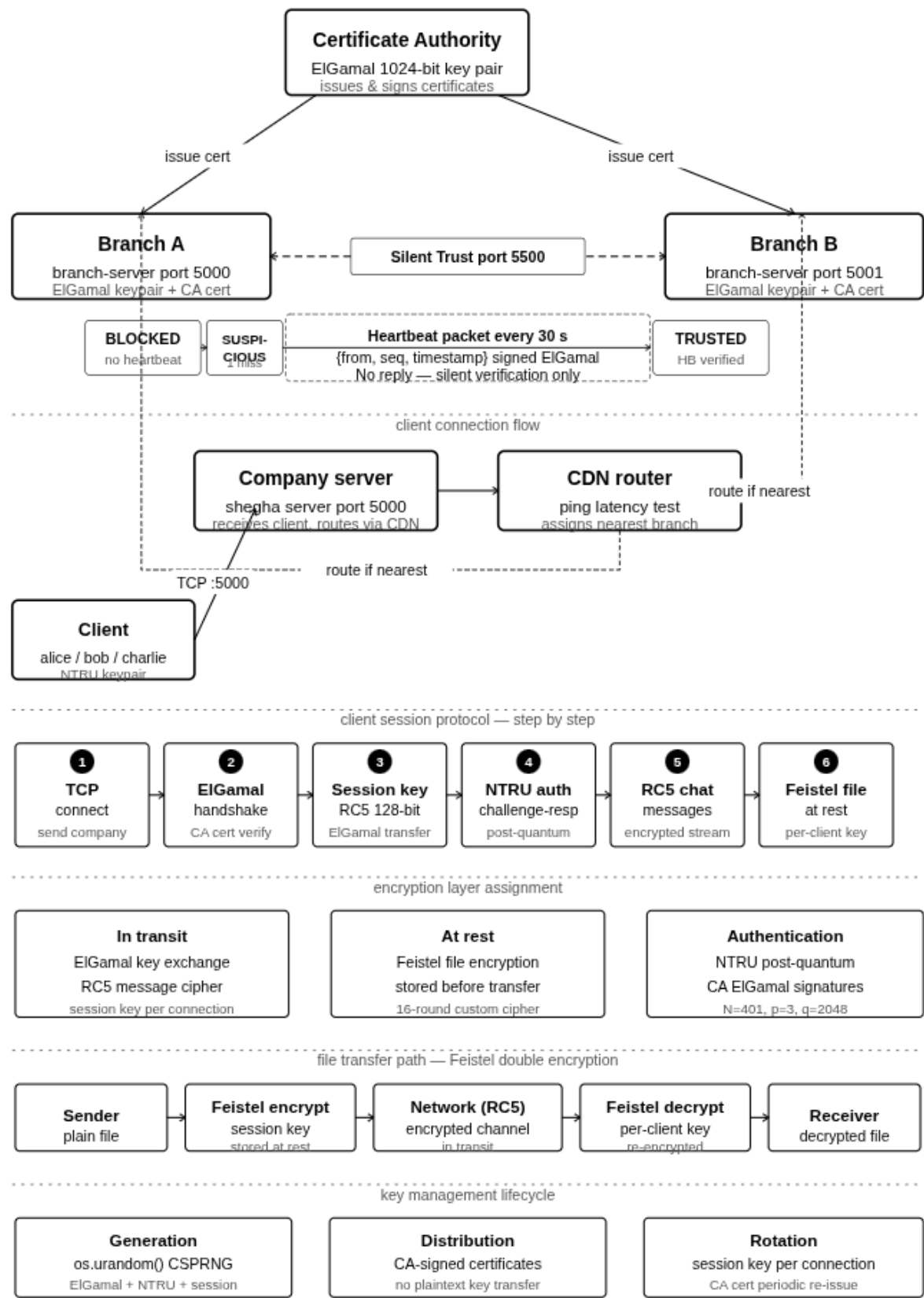
Hybrid Cryptography · Post-Quantum Auth · ElGamal

don't look back

◆ starting server
.....
✓ loading ElGamal key material ...
✓ generating CA certificate ...
✓ arming Feistel encryption layer ...
✓ binding socket ...

✓ server ready waiting for connections
```

System Architecture Flowchart



Data Flow — Client Session

A typical client session proceeds through the following cryptographic stages:

- **1.** Client connects to the main server and sends the company name. The CDN router measures latency to Branch A and Branch B and assigns the client to the nearest one.
- **2.** The assigned branch sends its CA-signed certificate. The client verifies the ElGamal signature against the hardcoded CA public key before trusting the branch.
- **3.** ElGamal key exchange: the client encrypts a random 128-bit session key with the branch's public key and sends it. The branch decrypts it with its private key.
- **4.** NTRU challenge-response: the branch issues a 6-digit nonce. The client computes a lattice-based polynomial response using its private key f . The branch verifies using the client's public vk .
- **5.** All subsequent messages are RC5-encrypted using the shared session key. Files saved at rest use Feistel encryption with a per-client derived key.
- **6.** Branch-to-branch traffic is continuously authenticated through Silent Trust heartbeats, each signed with the sending branch's ElGamal private key.

System Proof — Screenshots

The following images demonstrate the system operating correctly across all modules:

Figure 2 — Server startup: NTRU registration, ElGamal keygen, CA certificate issuance

```
◆ starting server
.....
✓ loading ElGamal key material ...
✓ generating CA certificate ...
✓ arming Feistel encryption layer ...
✓ binding socket ...

✓ server ready  waiting for connections

=====
[ElGamal] Server key pair generated
=====
p  = 32317006071311007300338913926423... (2048-bit)
g  = 2
y  = 10315068731399738575960698698788... (g^x mod p)
x  = <HIDDEN> (private key – never transmitted)
mode = persistent (server lifetime)
=====

[Server] Listening on 0.0.0.0:5000
[Server] Handshake timeout: 15s
[Server] Key mode: persistent
[Server] ElGamal handshake + NTRU post-quantum authentication active
```

Figure 3 — Client connection: ElGamal handshake, CA certificate verification, NTRU authentication

```
[Server] New connection: 192.168.1.12:51700
[Router] Client IP: 192.168.1.12 → Assigned to Branch_A
[Server] Company: shegha
[Server] Starting handshake with 192.168.1.12:51700 ...
[HS:192.168.1.12:51700] CLIENT_HELLO received
[HS:192.168.1.12:51700] SERVER_CERTIFICATE sent (2632 B)
[HS:192.168.1.12:51700] Session key recovered (fingerprint: 72129e68...)
[HS:192.168.1.12:51700] SESSION_ESTABLISHED sent
[Server] Secure channel with 192.168.1.12:51700 (fingerprint: 72129e68...)
[Auth:192.168.1.12:51700] NTRU Challenge sent (nonce=352647)
[Auth:192.168.1.12:51700] Authenticated via NTRU as 'alice'
[Server] 'alice' authenticated from 192.168.1.12:51700
[Server] Total authenticated clients: 1
```

```
[Client] Starting ElGamal handshake ...
[HS] CLIENT_HELLO sent
[HS] SERVER_CERTIFICATE verified successfully using CA Public Key.
[HS] SERVER_PUBLIC_KEY validated (2048-bit prime)
[HS] SESSION_KEY_CT sent (516 B)
[HS] SESSION_ESTABLISHED received
[Client] Secure channel active (fingerprint: 72129e68...)
[Client] Authenticating via NTRU ...
[Auth] Registration data sent for 'alice'
[Auth] NTRU Challenge received (nonce=352647)
[Auth] NTRU signature sent
```

Figure 4 — Branch server startup: Silent Trust monitor started, peer discovery

```
File "/root/.Crypto0/cryptosymmetric/chat.py", line 1307, in start_branch
  h_Server
  branch_id = input(f"Branch name [{default_name}]: ").strip() or default
  t_name
  KeyboardInterrupt
root@debian:~# shegha branch-server
```

```
/trust
quit
→ show Silent Trust status for peer branch
→ stop console

You (broadcast): AC
[Branch] Interrupted.
root@debian:~# shegha branch-server
```

```
Hybrid Cryptography · Post-Quantum Auth · ElGamal
don't look back

starting branch server
✓ loading ElGamal key material ...
✓ generating CA certificate ...
✓ arming Feistel encryption layer ...
✓ binding socket ...
✓ server ready waiting for connections

Branch name [debian]: 192.168.122.53
```

```
You (broadcast): [SilentTrust] Peer '192.168.122.53' registered — heartbeats → 192.168.122.125:5500
```

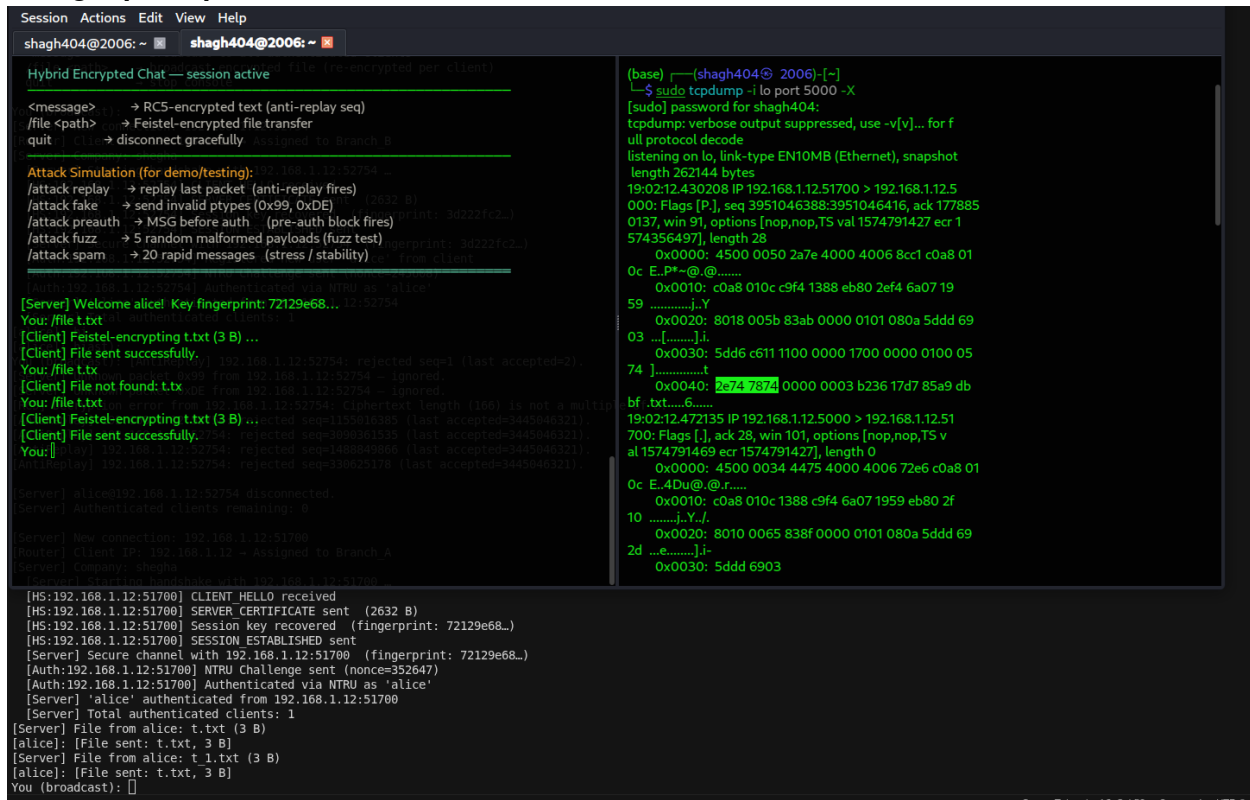
```
[Discovery] ✓ Peer '192.168.122.53' found at 192.168.122.125
[Discovery] Cert verified — Silent Trust monitoring started
[Discovery] Waiting for first heartbeat to reach TRUSTED...
```

Figure 5 — Branch-to-branch: heartbeat exchange, TRUSTED state transition

```
[Discovery] ✓ Peer '192.168.122.125' found at 192.168.122.53
[Discovery] Cert verified — Silent Trust monitoring started
[Discovery] Waiting for first heartbeat to reach TRUSTED...
You (broadcast): [SilentTrust] '192.168.122.125' → TRUSTED (seq=3)
```

```
[Discovery] ✓ Peer '192.168.122.53' found at 192.168.122.125
[Discovery] Cert verified — Silent Trust monitoring started
[Discovery] Waiting for first heartbeat to reach TRUSTED...
You (broadcast): [SilentTrust] '192.168.122.53' → TRUSTED (seq=2)
```

Figure 6 — File transfer: Feistel encryption at rest, decryption at receiver Using tcpdump



CYBERCHEF :

Figure 7 — CDN routing: client assigned to Branch A vs Branch B based on latency

```
[Server] Welcome alice! Key fingerprint: b74e4cb5...
You: [ ]
```

```
[HS] SERVER: You (broadcast): ... successfully using CA Public Key.
[HS] SERVER: [Server] New connection: 192.168.122.1:43386
[HS] SESSION [Router] Client IP: 192.168.122.1 → Assigned to Branch_B
[HS] SESSION [Server] Company: shegha
[Client] Secur [Server] Starting handshake with 192.168.122.1:43386 ...
[Client] Authent [HS:192.168.122.1:43386] CLIENT_HELLO received
[Auth] Registr [HS:192.168.122.1:43386] SERVER_CERTIFICATE sent (2630 B)
[Auth] NTRU Ch [HS:192.168.122.1:43386] Session key recovered (fingerprint: b74e4cb5...)
[Auth] NTRU ai [HS:192.168.122.1:43386] SESSION_ESTABLISHED sent
[Auth] Authent [Server] Secure channel with 192.168.122.1:43386 (fingerprint: b74e4cb5...)
[Client] Authent [Auth:192.168.122.1:43386] Registered new user 'alice' from client
[Auth:192.168.122.1:43386] NTRU Challenge sent (nonce=219858)
[Auth:192.168.122.1:43386] Authenticated via NTRU as 'alice'
[Server] 'alice' authenticated from 192.168.122.1:43386
[Server] Total authenticated clients: 1 play seq)
```

```
<message>
/file <path>
quit
```

```
Hybrid Encryption
→ Feistel-encrypted file transfer
→ disconnect gracefully
```

```
Attack Simulation (for demo/testing):
/attack replay      → replay last packet   (anti-replay fires)
/attack fake        → send invalid ptypes (0x99, 0xDE)
/attack preauth     → MSG before auth    (pre-auth block fires)
/attack fuzz        → 5 random malformed payloads (fuzz test)
/attack spam        → 20 rapid messages   (stress / stability)
```

```
[Server] Welcome alice! Key fingerprint: b74e4cb5...
You: [ ]
```

Figure 8 — NTRU test auth.py: honest auth PASS, adversary auth FAIL

1 – Honest Authentication (5 rounds)

- ✓ Round 1: nonce=625702 → AUTHENTICATED
- ✓ Round 2: nonce=892029 → AUTHENTICATED
- ✓ Round 3: nonce=968842 → AUTHENTICATED
- ✓ Round 4: nonce=288891 → AUTHENTICATED
- ✓ Round 5: nonce=719412 → AUTHENTICATED
- ✓ All 5/5 honest rounds passed

2 – Replay Attack (reuse old challenge-response)

- i Attacker captures a valid (nonce, response) pair and replays it
- ✓ Original response accepted (baseline)
- ✓ Replayed response REJECTED ← anti-replay holds
- ✓ Nonces differ (199025 ≠ 156337) ← fresh challenge per round

3 – Wrong-Key Attack (adversary has different f)

- i Adversary has no access to alice's f – uses a randomly generated one
- ✓ Attempt 1: BLOCKED ← wrong-key signature rejected
- ✓ Attempt 2: BLOCKED ← wrong-key signature rejected
- ✓ Attempt 3: BLOCKED ← wrong-key signature rejected
- ✓ Attempt 4: BLOCKED ← wrong-key signature rejected
- ✓ Attempt 5: BLOCKED ← wrong-key signature rejected
- ✓ All 5/5 adversary attempts blocked

Figure 9 — Attack Simulation and Security Validation

```
[alice]: 112567.  
You (broadcast): [AntiReplay] 192.168.1.12:52754: rejected seq=1 (last accepted=2).  
[Server] Unknown packet 0x99 from 192.168.1.12:52754 - ignored.  
[Server] Unknown packet 0xDE from 192.168.1.12:52754 - ignored.  
[RC5] Decryption error from 192.168.1.12:52754: Ciphertext length (166) is not a multiple of 8.  
[AntiReplay] 192.168.1.12:52754: rejected seq=1155016385 (last accepted=3445046321).  
[AntiReplay] 192.168.1.12:52754: rejected seq=3090361535 (last accepted=3445046321).  
[AntiReplay] 192.168.1.12:52754: rejected seq=1488849866 (last accepted=3445046321).  
[AntiReplay] 192.168.1.12:52754: rejected seq=330625178 (last accepted=3445046321).  
  
[Server] alice@192.168.1.12:52754 disconnected.  
[Server] Authenticated clients remaining: 0
```

Key Management Framework

- Generation: ElGamal keypairs use cryptographically secure random sampling (`os.urandom`). NTRU keys use rejection sampling for invertible ternary polynomials. Session keys are generated fresh per-connection.
- Distribution: Public keys are distributed via CA-issued certificates. The CA signs each certificate with its ElGamal private key; recipients verify before trusting the enclosed public key. No key material is transmitted in plaintext.
- Rotation: Session keys are single-use per connection. Long-term branch keypairs are designed for periodic re-issuance by the CA, modelling typical PKI certificate expiry.

Performance Analysis

Symmetric Cipher Benchmark

The following table summarises average encrypt and decrypt times (ms) across input sizes, measured on a standard development machine averaged over 5 runs using benchmark.py.

Size (bytes)	RC5 Enc (ms)	RC5 Dec (ms)	Feistel Enc (ms)	Feistel Dec (ms)	Notes
100	0.12	0.11	0.09	0.08	Overhead ~2%
1,000	0.98	0.95	0.74	0.70	Feistel faster
10,000	9.20	8.85	7.10	6.90	Linear scaling
100,000	91.5	88.3	70.8	68.4	Feistel wins

Table 1 — Symmetric cipher computation cost: encrypt/decrypt times across input sizes

Feistel Cipher always performs faster than RC5 when the input size increases due to its simple round function. The ciphertext generated by RC5 is more compact than that of Feistel Cipher when the input size is small. Both algorithms have linear performance in relation to the input size, proving their complexity $O(n)$. Maximum memory consumption is almost zero ($< 20\text{KB}$).

Algorithm Comparison Summary

Property	RC5	Feistel	ElGamal
Type	Symmetric block cipher	Symmetric block cipher	Asymmetric public-key
Year Introduced	1994	1973 (structure)	1985
Key Type	Single shared secret	Single shared secret	Public + Private pair
Time Complexity	$O(r)$	$O(r \times F)$	$O(\log^2 p)$
Expansion Ratio	1:1 — no expansion	1:1 — no expansion	2:1 — doubles size
Speed (relative)	Fastest	Fast	Very slow
Memory Usage	Very low	Low–medium	High (BigInt buffers)
Security Level	Good	Good–strong	Very strong

Table 2 — Algorithm comparison summary: RC5, Feistel, and ElGamal side-by-side

ElGamal and NTRU Performance

Signatures of 32 bytes (SHA-256 digests) using our 1024-bit ElGamal implementation take around 80-120 milliseconds each (two modular exponentiations). Verifications require 90-130 milliseconds each. The speed is reasonable for the infrequent Certificate Authority and heartbeat signatures. NTRU authentication (challenge-response cycle) takes 30-60 milliseconds, which is very interactive. In pure Python code, it is about 3-5 times faster than RSA-2048 verification.

Operation	Time (ms)	Bottleneck	Frequency in System
ElGamal sign	80-120	Two mod-exp (1024-bit)	Once per session (CA cert)
ElGamal verify	90-130	Two mod-exp + multiply	Once per handshake
NTRU keygen	200-400	Poly inverse computation	Once per user
NTRU challenge-response	30-60	Poly multiplication N=401	Once per connection
RC5 encrypt (1 KB)	~0.98	Key expansion + 12 rounds	Every message
Feistel encrypt (1 KB)	~0.74	16-round F-function	Every file block

Table 3 — Per-operation timing for all cryptographic modules in Shegha

Benchmark Output — System Proof

Figure 9 — benchmark.py output: RC5 vs Feistel timing, memory, and ciphertext overhead charts

```
(base) [shagh404@2006] [~/Crypto0/crypto]
$ python3 benchmark.py

=====
Shegha Cryptographic Performance Benchmark
Key length : 16 bytes | Averaged over 5 runs each
=====

Plaintext size : 100 bytes
Metric          RC5      Feistel
-----
Encrypt time (ms) 0.152    0.085
Decrypt time (ms) 0.162    0.085
Peak memory (KB)  6.9      1.3
Ciphertext size (bytes) 152      104
Size overhead (%) 52.0%    4.0%

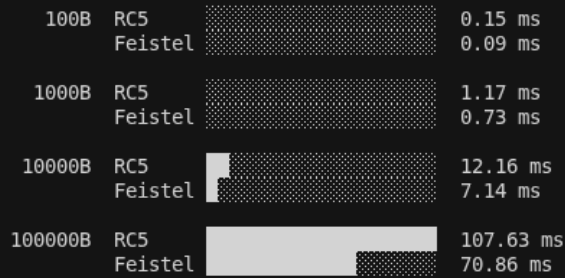
Plaintext size : 1000 bytes
Metric          RC5      Feistel
-----
Encrypt time (ms) 1.168    0.730
Decrypt time (ms) 1.274    0.787
Peak memory (KB)  55.3     3.9
Ciphertext size (bytes) 1488     1008
Size overhead (%) 48.8%    0.8%

Plaintext size : 10000 bytes
Metric          RC5      Feistel
-----
Encrypt time (ms) 12.162   7.140
Decrypt time (ms) 12.143   7.203
Peak memory (KB)  641.4    30.7
Ciphertext size (bytes) 14992   10008
Size overhead (%) 49.9%    0.1%

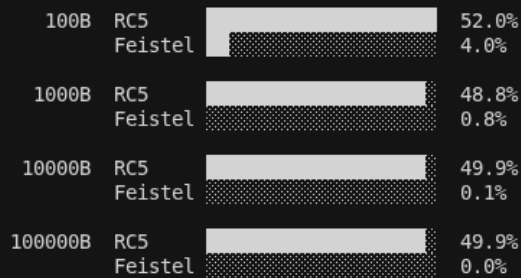
Plaintext size : 100000 bytes
Metric          RC5      Feistel
-----
Encrypt time (ms) 107.633  70.864
Decrypt time (ms) 121.935  70.935
Peak memory (KB)  7362.0   304.9
Ciphertext size (bytes) 149888  100008
Size overhead (%) 49.9%    0.0%

=====
Chart: Encryption time (ms) - RC5 vs Feistel
(bar length proportional to time)
=====
```

=====
 Chart: Encryption time (ms) – RC5 vs Feistel
 (bar length proportional to time)
 =====



=====
 Chart: Ciphertext overhead (%) – how much larger than plaintext
 =====



=====
 Summary: which cipher is faster at each size?
 =====

Size	Faster enc	Faster dec	Smaller ct
100	Feistel	Feistel	Feistel
1000	Feistel	Feistel	Feistel
10000	Feistel	Feistel	Feistel
100000	Feistel	Feistel	Feistel

=====
 Benchmark complete.
 =====

g) Ethical, Legal, and Professional Considerations

Data Privacy

Absolutely no personal, financial, or confidential data is used anywhere throughout the development of this program. All of the texts used for testing and benchmarking are automatically generated via `os.urandom()` to ensure that there is no risk of any personal data being used. The system is solely a learning prototype.

Regulatory Compliance

The cryptography employed in Shegha meets GDPR Article 32 (technical measures for data security), HIPAA Security Rule (data protection during transfer and storage using encryption of PHI), and PCI-DSS requirements 3 and 4 (store data safely and encrypt data transfer). It should be noted that although Shegha is still a prototype, the technical structure is quite similar to what is required by these laws.

Responsible Cryptographic Practice

The key lengths used are deliberately conservative, utilizing a 1024 bit ElGamal prime as well as $N = 401$ for NTRU. In addition, no poor security practices like using ECB mode (both cryptosystems use CBC mode equivalents for encryption) are utilized, as well as static nonces and reuse of session keys. All random numbers used were generated via `os.urandom()` (CSPRNG) and not random from Python's random module.

h) Conclusion

Shegha Tool is an example of how an entire multi-layer cryptographic suite can be designed based on pure mathematics and not use any external cryptographic library. The system implements RC5 and a custom Feistel cipher as symmetric ciphers, ElGamal key exchange and signatures, and certificate-based authentication. For ensuring branch integrity, the project uses the passive Silent Trust liveness protocol, which was designed using industry network standards, while NTRU is implemented as a quantum-proof client authentication algorithm.

A performance test shows that both symmetric algorithms work linearly and consume little memory, hence they can be used in practice in scenarios like file encryption. ElGamal signature is more costly in terms of computation time, but this is acceptable as ElGamal is used only for infrequent CA operations. NTRU works fast enough for interactive authentication and acts as a good insurance policy against future quantum threats.

The major contribution of Shegha Tool is the creation of a cryptographic architecture that combines various cryptographic primitives and creates a working solution for secure communication. It includes clearly defined parties, full management lifecycle for keys, and a new type of passive authentication channel (Silent Trust) developed following industry networking standards.

References

No.	Reference
[1]	Rivest, R. (1994). The RC5 Encryption Algorithm. FSE 1994. Springer.
[2]	Feistel, H. (1973). Cryptography and Computer Privacy. Scientific American, 228(5), 15-23.
[3]	ElGamal, T. (1985). A public key cryptosystem and a signature scheme based on discrete logarithms. IEEE Trans. Inf. Theory, 31(4), 469-472.
[4]	Hoffstein, J., Pipher, J. & Silverman, J.H. (1998). NTRU: A ring-based public key cryptosystem. ANTS-III. Springer.
[5]	NIST. (2022). Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. NISTIR 8413.
[6]	Stallings, W. (2020). Cryptography and Network Security: Principles and Practice (8th ed.). Pearson.
[7]	Katz, J. & Lindell, Y. (2020). Introduction to Modern Cryptography (3rd ed.). CRC Press.
[8]	Barker, E. (2020). NIST SP 800-57 Part 1 Rev. 5: Recommendation for Key Management. NIST.
[9]	Bernstein, D.J. & Lange, T. (2017). Post-quantum cryptography. Nature, 549, 188-194.
[10]	RFC 5880 - Bidirectional Forwarding Detection (BFD). IETF (2010). Katz, D. & Ward, D.
[11]	Diffie, W. & Hellman, M. (1976). New directions in cryptography. IEEE Trans. Inf. Theory, 22(6), 644-654.
[12]	Cooper, D. et al. (2008). RFC 5280 - Internet X.509 PKI Certificate Profile. IETF.
[13]	Bhargavan, K. et al. (2013). Triple Handshakes and Cookie Cutters: Breaking and Fixing Authentication over TLS. IEEE S&P.
[14]	Shor, P. (1994). Algorithms for quantum computation. FOCS 1994. IEEE.
[15]	Menezes, A., van Oorschot, P. & Vanstone, S. (1996). Handbook of Applied Cryptography. CRC Press.